Helpyard dashboard modernization plan

Prepared: 17 October 2026

Scope: Local Helpyard dashboard, Python bridge, and safe resource/dependency handling.

Objective

Make the full dashboard clearer, more professional, configurable, and easier to operate while preserving local-first behavior, explicit approvals, and the existing security boundaries. Completion means all navigation sections and tested behaviors work; it does not mean enabling currently disabled features without a separate reviewed implementation.

Current project baseline

- A single-page HTML/CSS/JavaScript dashboard with 12 sections: Overview, Projects, Workbench, Processes, Services, Terminal, Local AI, Cloud AI, MCP, Skills, Security, and Settings.
- A Python 3.10+ bridge using only the standard library and listening on IPv4 loopback.
- Existing theme support, live CPU/memory charts, disk capacity, local Ollama integration, and confirmation gates for system mutations.
- No pip packages, frontend build step, or third-party UI assets are required.
- Arbitrary shell execution and MCP server startup are intentionally disabled. Skills are informational examples, not executable downloads.

Step-by-step implementation roadmap

Phase 1 — Audit and safety baseline (complete)

1. Map all dashboard navigation pages, bridge API responsibilities, tests, and documented limitations.
2. Preserve one-time bootstrap, in-memory bridge token handling, session-only cloud API keys, loopback restrictions, and explicit action confirmations.
3. Verify dashboard routes remain implemented by the bridge and navigation/page pairs remain complete.

Phase 2 — Visual system and responsive dashboard (first implementation complete)

1. Refine spacing, typography, panel depth, navigation, tables, and responsive layout using native CSS.
2. Preserve dark/light appearance and provide four fixed, validated accent choices.
3. Avoid remote fonts, CDN code, and downloaded UI frameworks so the local dashboard remains offline-capable.

Phase 3 — Overview and contextual guidance (first implementation complete)

1. Keep CPU, memory, uptime, process count, disk capacity, and CPU/memory history in the overview.
2. Label history accurately: five-second samples while Overview is open, limited to the current page session, and not stored as long-term history.
3. Add a safe, page-specific guidance portal to all 12 sections, with a relevant in-dashboard link.
4. Let users disable contextual tips without changing any action permissions.

Phase 4 — Customization and AI/resource surfaces (initial controls complete; broader work remains)

1. Save theme/accent/tip visibility in browser preferences; never persist bridge tokens or provider API keys.
2. Continue improving Workbench, Local AI, Skills, and MCP status UX only against capabilities implemented by the bridge.
3. Keep skills and MCP entries informational until a reviewed registry, provenance, version/hash verification, and explicit approval workflow exist.
4. Do not add a generic download-and-execute path.

Phase 5 — Validation and release readiness (ongoing)

1. Run dashboard structural/security checks and Python bridge unit tests.
2. Test all pages, preference persistence, fresh browser storage, unavailable bridge/Ollama, narrow screens, and keyboard focus.
3. Verify that actions requiring approval still ask, shell execution remains disabled, and no token/key enters persistent browser storage.
4. Separately qualify browser accessibility, Windows versions, dependency/license provenance, and real-world release security before calling the product release-ready.

Resource and dependency policy

- Current Python requirements: Python 3.10+ standard library only; no pip package is needed.
- Current web interface: browser-native HTML/CSS/JavaScript/canvas; no UI library or external asset is required.
- Ollama is an optional separately installed local service, not a project pip dependency.
- The resource catalog records the rationale, candidate package evaluation criteria, and safe download policy. No external package or asset was downloaded for this change.
- Any future download requires explicit purpose, official source/version/license review, integrity verification, source review, and documentation. Downloaded skills or MCP servers must never execute automatically.

First implementation delivered

- Professionalized the visual hierarchy and responsive navigation with native CSS.
- Added fixed, validated accent-color customization alongside the existing light/dark theme toggle.
- Added section-specific, informational safe tips for all 12 pages and a setting to hide them.
- Kept user preferences non-secret and documented the short-lived nature of graph history.
- Added regression assertions for page coverage, customization controls, and secret-storage safeguards.

Definition of done

All dashboard navigation sections render; each has applicable contextual guidance; customization works and persists only non-secret preferences; bridge and dashboard checks pass; no unreviewed dependency is introduced; and currently disabled actions remain disabled. Any broader feature—long-term telemetry, executable skills, MCP process lifecycle, named terminal tasks, or downloadable extensions—requires its own design, approval, implementation, and validation plan.